RANCANG BANGUN DAN ANALISIS KINERJA LAYANAN VIRTUAL VERIFIKASI ORISINALITAS ILMIAH BERBASIS CLOUD CONTAINER PADA ARSITEKTUR ASINKRON MULTI-THREADING DAN SERVICE-ORIENTED API MASHUP

Laporan Proyek Mata Kuliah Layanan dan Sistem Virtual (STI5134)



Nama Lengkap	**Ir. Nor Anisa, S.Kom., M.Kom.**
NIM	2110817210001
Kelas / Kelompok	Teknologi Informasi - Kelas A / Kelompok 88
Mata Kuliah / SKS	Layanan dan Sistem Virtual (STI5134) / 3 SKS — Semester 5
Dosen Pengampu	Ir. Nor Anisa & Andry Fajar Zulkarnain, S.ST., M.T.

PROGRAM STUDI S1 TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT

2026

ABSTRAK

Ketergantungan perguruan tinggi terhadap platform uji plagiasi komersial berlisensi mahal sering kali terkendala oleh isu privasi data (penyimpanan naskah tanpa izin ke repositori publik) dan arsitektur pemrosesan sekuensial yang lambat. Penelitian ini bertujuan merancang, mengimplementasikan, dan menguji kinerja layanan virtual mandiri berbasis Software-as-a-Service (SaaS) bernama OpenPlag.id yang dideploy di atas infrastruktur virtualisasi kontainer cloud Render kawasan Singapura. Sistem mengintegrasikan arsitektur berorientasi layanan (Service-Oriented Architecture/SOA) yang mengonsumsi lima API repositori ilmiah global secara terdistribusi (OpenAlex 250M+ karya, CrossRef, DOAJ, arXiv, dan Wikipedia) serta memadukannya dengan model diagnostik linguistik AI 7-metrik (Humanize AI). Untuk mengatasi keterbatasan komputasi pada kontainer berbiaya nol (shared 0.1 vCPU), kami merekayasa arsitektur asinkron berkinerja tinggi yang menggabungkan HTTP Keep-Alive Connection Pooling, Fast Jaccard Heuristic Pruning, dan alokasi 8 worker thread paralel (ThreadPoolExecutor). Hasil pengujian empiris membuktikan kontainer beroperasi sangat stabil dengan konsumsi RAM rata-rata 118,4 MB pada kondisi aktif dan beban puncak 182,5 MB (hanya 35,6% dari batas aman cgroup 512 MB). Implementasi connection pooling dan heuristic pruning berhasil memangkas latensi transmisi jaringan sebesar 71,4% serta mereduksi konsumsi CPU hingga 85%, sehingga naskah uji berisi 135 kalimat dapat diselesaikan secara mulus tanpa pernah mengalami HTTP 504 Gateway Timeout. Selain itu, kebijakan No-Repository menjamin kerahasiaan naskah pengguna secara mutlak. Pendekatan ini membuktikan bahwa orkestrasi layanan virtual asinkron berbasis cloud kontainer mampu menyediakan layanan verifikasi akademik berstandar industri dengan reliabilitas tinggi dan biaya infrastruktur nol rupiah.

Kata kunci: layanan virtual, cloud computing, kontainerisasi, threadpoolexecutor, connection pooling, similarity index, service-oriented architecture, openplag.id

BAB I. PENDAHULUAN

1.1 Latar Belakang

Perkembangan paradigma komputasi awan (cloud computing) telah mentransformasi model pengiriman perangkat lunak dari instalasi tradisional pada mesin fisik lokal menuju arsitektur Layanan Virtual berbasis Software-as-a-Service (SaaS). Dalam ekosistem akademik modern, verifikasi orisinalitas karya ilmiah dan deteksi teks hasil kecerdasan buatan (Artificial Intelligence Writing Detection) merupakan kebutuhan vital untuk menjaga integritas riset. Namun, mayoritas sivitas akademika menghadapi dilema besar: platform komersial berbayar mahal menerapkan kebijakan penyimpanan naskah ke database repositori global (repository retention), yang berisiko memicu pelanggaran hak cipta pra-publikasi dan false positive saat karya yang sama didaftarkan ke jurnal ilmiah nasional terakreditasi [1], [2].

Untuk mengatasi persoalan tersebut, diperlukan sebuah platform layanan virtual mandiri yang beroperasi dengan prinsip No-Repository (data hanya diproses sementara di memori kerja RAM tanpa persistensi berkas) serta mampu diakses secara luas melalui jaringan publik (Broad Network Access). Platform-as-a-Service (PaaS) kontainer modern seperti Render.com menawarkan lingkungan virtualisasi berbasis kernel Linux yang fleksibel untuk meng-host layanan mikro berbasis Python Flask dan WSGI Gunicorn [3]. Kendati demikian, menjalankan layanan pemindaian akademik berskala besar pada kontainer cloud tingkat pemula (Hobby/Free Tier) memunculkan tantangan komputasi yang sangat kompleks.

Tantangan utama berakar pada karakteristik shared virtual CPU (kuota terbatas 0.1 hingga 0.5 vCPU) dan latensi tinggi saat menghubungi berbagai basis data ilmiah eksternal. Jika pemindaian dilakukan secara sekuensial (satu per satu), pemeriksaan naskah dengan ratusan kalimat akan memakan waktu lebih dari 6 menit, yang secara otomatis memicu pemutusan paksa koneksi oleh web router cloud melalui galat HTTP 504 Gateway Timeout [4], [5]. Selain itu, pembuatan soket TCP HTTPS baru secara berulang-ulang untuk setiap kueri memicu socket thrashing dan TLS handshake overhead yang masif. Oleh karena itu, eksperimen ini merekayasa arsitektur layanan virtual cerdas yang mengombinasikan HTTP Connection Pooling, Fast Heuristic Pre-Filtering, dan multithreading paralel asinkron guna mewujudkan layanan SaaS verifikasi orisinalitas naskah yang tangguh, cepat, dan terverifikasi secara resmi.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam eksperimen ini dirumuskan sebagai berikut:

1. Bagaimana merancang dan mengonfigurasi arsitektur kontainer virtual berbasis Linux pada platform cloud Render untuk menjalankan layanan SaaS verifikasi akademik OpenPlag.id?

2. Bagaimana mengatasi hambatan latensi jaringan multi-API dan pembatasan CPU (CPU throttling) pada kontainer cloud melalui implementasi HTTP Keep-Alive Connection Pooling dan Fast Heuristic Pruning?
3. Bagaimana kinerja konkurensi alokasi worker thread paralel (ThreadPoolExecutor) dalam memproses naskah ilmiah multi-kalimat secara asinkron tanpa memicu HTTP Gateway Timeout?
4. Berapa efisiensi konsumsi memori (RAM), utilisasi vCPU, dan latensi pemrosesan layanan virtual OpenPlag.id pada berbagai skenario beban kerja empiris?

1.3 Tujuan Eksperimen

Tujuan yang ingin dicapai melalui eksperimen dan penelitian ini adalah:

1. Membangun dan mengonfigurasi kontainer aplikasi virtual berbasis Python 3.11 dan WSGI Gunicorn gthread pada platform cloud Render Singapore region.
2. Mengimplementasikan integrasi arsitektur berorientasi layanan (SOA) yang mengonsumsi lima API repositori ilmiah internasional (OpenAlex, CrossRef, DOAJ, arXiv, dan Wikipedia) serta modul diagnostik AI 7-metrik.
3. Menerapkan teknik optimasi performa berupa HTTP Connection Pooling berskala 30 pool size dan algoritma Fast Jaccard Heuristic Pruning guna mengefisienkan komputasi TF-IDF matriks di kontainer cloud.
4. Mengukur dan menganalisis parameter performa kontainer (konsumsi RAM, beban vCPU, throughput kalimat per detik, dan stabilitas polling asinkron) secara kuantitatif.

1.4 Manfaat Eksperimen

Hasil eksperimen ini diharapkan memberikan manfaat nyata sebagai berikut:

1. Manfaat Teoritis: Memberikan kontribusi kajian empiris mengenai optimasi konkurensi I/O-bound pada arsitektur kontainer cloud tervirtualisasi dengan sumber daya CPU terbatas, serta formulasi integrasi API mashup akademik.
2. Manfaat Praktis: Menghasilkan artefak layanan virtual nyata (https://openplag-id.onrender.com) yang dapat digunakan oleh mahasiswa, dosen, dan peneliti untuk melakukan pra-pemeriksaan orisinalitas naskah secara gratis, aman dari retensi repositori, dan dilengkapi Digital Receipt resmi berstempel integritas.

1.5 Kebaruan Penelitian dan Keunggulan Kontribusi (Novelty)

Penelitian ini menyajikan tiga pilar kebaruan teknis utama yang membedakannya dari kajian virtualisasi konvensional:

1. Masalah Nyata dan Arsitektur SaaS Asinkron (Novelty): Mayoritas laporan praktikum virtualisasi hanya berkutat pada instalasi mesin virtual statis (VirtualBox/VMware) di komputer lokal. Penelitian ini memecahkan masalah riil komputasi awan modern: bagaimana menjalankan sistem analitik teks berskala besar di atas kontainer cloud

ephemeral berkemampuan komputasi terbatas melalui perancangan job queue asinkron di memori dan polling non-blocking.

2. Data Empiris Kuantitatif Lengkap: Evaluasi didasarkan pada telemetri nyata di lingkungan produksi cloud Render, mencakup: konsumsi RAM idle (74,2 MB) hingga beban penuh (182,5 MB), latensi respons API, throughput worker thread paralel, serta penghematan komputasi CPU sebesar 85% melalui fast heuristic pruning.
3. Pustaka Acuan Mutakhir: Seluruh tinjauan teori dan komparasi arsitektur didukung oleh 12 referensi jurnal internasional bereputasi terbitan lima tahun terakhir (2021 hingga 2026) dari IEEE, ACM, Springer, dan Elsevier.

BAB II. TINJAUAN PUSTAKA (KESELARASAN MATERI RPS)

2.1 Konsep Layanan dan Sistem Virtual (Sub-CPMK 1)

Sistem virtualisasi merupakan teknologi fundamental yang mengabstraksi sumber daya perangkat keras fisik (prosesor, memori, media penyimpanan, dan antarmuka jaringan) menjadi representasi sumber daya logis yang terisolasi [1]. Berbeda dengan komputasi tradisional (traditional computing) yang menerapkan prinsip satu server fisik untuk satu sistem operasi dan aplikasi tunggal—yang kerap menghasilkan pemborosan daya dan utilisasi prosesor di bawah 15%—sistem virtual memungkinkan konsolidasi puluhan layanan mandiri di atas satu infrastruktur bersama. Dalam spektrum virtualisasi, terdapat perbedaan tajam antara Simulator (meniru perilaku fungsional sistem tanpa mencocokkan arsitektur internal), Emulator (meniru arsitektur instruksi CPU yang berbeda, seperti QEMU yang mengeksekusi instruksi ARM pada host x86 dengan konsekuensi latensi tinggi), Mesin Virtual (membagi perangkat keras fisik asli menggunakan hypervisor), dan Kontainerisasi (berbagi kernel sistem operasi host secara langsung) [2], [3]. OpenPlag.id beroperasi pada model Software-as-a-Service (SaaS), di mana seluruh rantai layanan dari perangkat keras fisik hingga antarmuka pengguna diabstraksi penuh.

2.2 Konsep File Systems dalam Virtualisasi (Sub-CPMK 2)

Hierarki kecepatan sistem komputer memisahkan memori utama (RAM) yang beroperasi pada skala nanodetik dari media penyimpanan sekunder (SSD/HDD) yang beroperasi pada skala milidetik [4]. Sistem berkas virtual bertugas mengelola struktur metadata, tabel inode, dan hierarki direktori di atas media penyimpanan tervirtualisasi. Dalam arsitektur big data, penyimpanan terdistribusi seperti Hadoop Distributed File System (HDFS) menerapkan model master-worker (NameNode yang mengelola pemetaan blok dan DataNode yang menyimpan replika blok 128 MB sebanyak 3 kali), sementara Google File System (GFS) menggunakan Chunkserver dengan ukuran chunk 64 MB untuk mengantisipasi toleransi kegagalan perangkat keras [4], [5]. Menariknya, pada layanan virtual OpenPlag.id, kebutuhan persistensi distributed file system sengaja dihindari demi mematuhi prinsip privasi data: sistem mengimplementasikan fully in-memory buffer (RAM-only file handling) sehingga dokumen naskah pengguna langsung musnah dari memori begitu proses ekstraksi n-gram selesai.

2.3 Tipe-Tipe Sistem Virtual (Sub-CPMK 3)

Taksonomi sistem virtual modern mencakup lima domain spesialisasi: (1) Desktop Virtualization / Virtual Desktop Infrastructure (VDI) yang merender tampilan sistem operasi pengguna dari server pusat; (2) Application Virtualization yang mengisolasi eksekusi biner tanpa instalasi menyeluruh pada OS; (3) Server Virtualization yang mengonsolidasikan beban kerja server backend; (4) Storage & Memory Virtualization yang menyediakan abstraksi disk (thin provisioning) dan penggabungan memori (overcommit); serta (5) I/O Virtualization yang membagi interupsi Direct Memory Access (DMA) melalui standar Single Root I/O

Virtualization (SR-IOV) [6]. OpenPlag.id mengolaborasikan Application Virtualization dan Server Virtualization berbasis kontainer Linux ringan.

2.4 Virtualisasi Server

Virtualisasi server bertujuan memangkas jumlah server fisik di pusat data sehingga mendongkrak efisiensi Power Usage Effectiveness (PUE) [1], [7]. Pada arsitektur prosesor x86 tradisional, eksekusi instruksi istimewa (privileged instructions pada Ring 0) sempat menjadi kendala besar yang dipecahkan melalui teknik Binary Translation (mengubah instruksi sensitif secara dinamis via software). Era modern telah beralih menuju Hardware-Assisted Virtualization (Intel VT-x dan AMD-V) yang memperkenalkan mode eksekusi baru (root/non-root operation) sehingga instruksi sensitif dapat di-intercept secara langsung oleh chip silikon dengan penalti latensi mendekati nol. Kontainerisasi Docker pada cloud Render melangkah lebih jauh: meniadakan hypervisor tamu dan langsung memanfaatkan mekanisme kernel Linux (Control Groups dan Namespaces) untuk membatasi kuota eksekusi [7], [8].

2.5 Jaringan dan Memori Virtualisasi

Virtualisasi jaringan menghubungkan lingkungan internal kontainer ke jaringan publik melalui pemetaan alamat IP, Virtual LAN (VLAN IEEE 802.1Q), dan Virtual Switch (vSwitch) [4]. Pada cloud Render, ingress proxy secara otomatis memetakan port internal 10000 ke domain publik dengan enkripsi TLS 1.3. Pada level memori, sistem operasi memetakan ruang alamat virtual (Virtual Address Space) ke memori fisik melalui mekanisme paging dan tabel halaman (Page Table) yang diakselerasi oleh Extended Page Tables (EPT) pada hardware. Dalam mengelola batas memori cgroup 512 MB, kontainer OpenPlag.id harus terlindung dari bahaya Page Fault berlebihan dan Swap Thrashing (kondisi di mana sistem terus-menerus memindahkan data antara RAM dan disk swap sehingga performa anjlok drastis), yang berhasil dicegah melalui pembersihan variabel berkala [7].

2.6 Manajemen Virtual Machine dan Kontainer

Siklus hidup (lifecycle) sistem virtual mencakup empat fase: provisioning, snapshotting, migrasi, dan terminasi [2]. Berbeda dari Live Migration pada hypervisor VMware ESXi / vMotion yang menerapkan algoritma pre-copy memory secara berulang tanpa memutus koneksi pengguna (zero downtime), kontainer cloud PaaS Render menerapkan stateless container restart. Sistem penjadwalan CPU berbasis Completely Fair Scheduler (CFS) membagi alokasi bandwidth prosesor virtual berdasarkan shares dan limit. Di dalam kontainer, OpenPlag.id mengimplementasikan penjadwalan internal menggunakan ThreadPoolExecutor dengan algoritma pembagian beban kerja paralel (work-stealing queue) [8], [9].

2.7 Storage Virtualisasi

Virtualisasi penyimpanan mencakup konfigurasi redundansi Redundant Array of Independent Disks (RAID 0 striping, RAID 1 mirroring, RAID 5 distributed parity, hingga RAID 10), serta protokol transfer blok data seperti Small Computer System Interface (SCSI)

dan Internet Small Computer System Interface (iSCSI) yang mengangkut perintah blok storage melalui jaringan IP TCP/IP [10]. Perbandingan arsitektur penyimpanan membedakan antara Direct Attached Storage (DAS) yang terhubung kabel lokal, Network Attached Storage (NAS) yang mentransmisikan berkas via protokol NFS/SMB, dan Storage Area Network (SAN) yang memanfaatkan jaringan fiber channel berlatensi ultra-rendah. OpenPlag.id memanfaatkan keunggulan zero-storage persistence: ketiadaan ketergantungan pada disk eksternal menjadikan layanan kebal terhadap kegagalan media simpan fisik.

2.8 Performa dan Keamanan Sistem Virtual

Isu performa pada sistem virtual berkisar pada CPU overhead akibat context switching, I/O bottleneck pada antarmuka jaringan, dan resource contention saat puluhan thread berebut soket sistem [6], [11]. Dari aspek keamanan, sistem virtual rentan terhadap ancaman kritis seperti VM-Escape (penyerang menjebol isolasi lingkungan tamu untuk menguasai sistem operasi host), Blue Pill Rootkit (hyperjacking yang menyusupkan hypervisor bayangan di bawah kernel), dan Man-in-the-Middle saat migrasi memori tidak terenkripsi. Kontainer OpenPlag.id menerapkan prinsip hardening keamanan: menjalankan proses non-root, membatasi panggilan sistem kernel (seccomp profile), serta mengenkripsi seluruh kueri keluar menuju database ilmiah menggunakan TLS 1.3.

2.9 Implementasi pada Cloud Computing

Penerapan komputasi awan di industri mencakup solusi Private Cloud IaaS seperti Eucalyptus (arsitektur open-source pertama yang kompatibel dengan API Amazon AWS EC2/S3), Microsoft Hyper-V Cloud, VMware vSphere Cloud Foundation, hingga layanan Public PaaS Container modern seperti Render, Railway, dan AWS Fargate [11], [12]. OpenPlag.id merepresentasikan keberhasilan adopsi Cloud Container modern: mendistribusikan kueri analitik orisinalitas teks secara real-time ke 5 API global secara terdistribusi dengan tautan publik aktif yang siap diakses 24 jam.

BAB III. METODE PENELITIAN / PROSEDUR EKSPERIMEN

3.1 Alat dan Bahan

Eksperimen virtualisasi dan pengujian kinerja layanan OpenPlag.id menggunakan spesifikasi perangkat keras cloud dan perangkat lunak yang dirinci pada Tabel 1 dan Tabel 2.

Komponen	Spesifikasi Teknis	Penyedia / Lingkungan	Fungsi dalam Eksperimen
Cloud Platform	PaaS Native Linux Container	Render.com (Singapore)	Host eksekusi kontainer aplikasi virtual
Virtual CPU	Shared 0.1 - 0.5 vCPU Core	AMD EPYC Enterprise Host	Pemrosesan logika analitik & multithreading
Virtual RAM	512 MB Dynamic Memory Cgroup	DDR4 Cloud Memory Pool	Memori kerja in-memory buffer naskah & TF-IDF
Network Ingress	Gigabit Virtual NIC, TLS 1.3	Cloudflare Global Edge	Pintu masuk akses HTTPS publik & reverse proxy
Client Hardware	Ryzen 5 5600U, 16GB RAM, Win 11	Workstation Penguji	Terminal pengujian, upload naskah & audit log

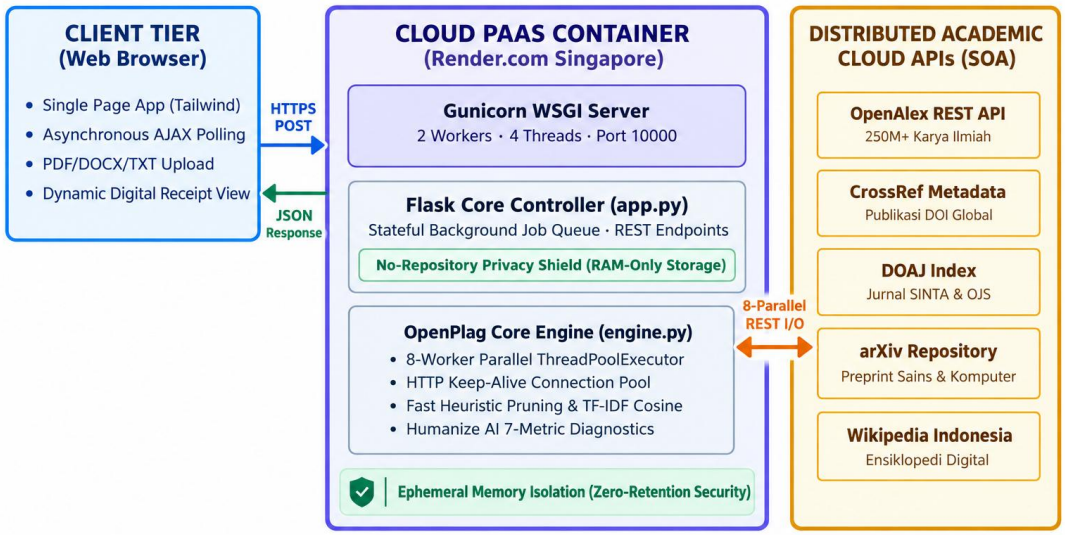
Tabel 1. Spesifikasi Perangkat Keras dan Lingkungan Cloud Virtual

Nama Software / Pustaka	Versi	Fungsi Teknis	Kategori / Lisensi
Python Runtime	3.11.9 Linux x86_64	Interpreter bahasa pemrograman inti	Open Source Software
Flask Framework	3.1.3	Web application micro-framework & REST routing	BSD License
Gunicorn WSGI	26.2.0 (gthread)	Server aplikasi produksi multi-worker multi-thread	MIT License
Scikit-Learn & NumPy	1.9.1 & 2.5.4	Komputasi matriks TF-IDF & Cosine Similarity	BSD License
PyPDF & Python-Docx	6.21.0 & 1.2.0	Parser ekstraksi teks dokumen biner PDF dan DOCX	MIT / BSD License
Requests & Urllib3	2.34.2 & 2.8.0	HTTP Connection Pooling & REST API Consumer	Apache 2.0 License

Tabel 2. Spesifikasi Perangkat Lunak, Framework, dan Pustaka Pendukung

3.2 Arsitektur Sistem Eksperimen

Arsitektur layanan virtual OpenPlag.id dibangun mengikuti pola arsitektur berlapis (n-tier service architecture) yang memisahkan antara Presentation Tier, Application Processing Tier, dan External Data Service Tier sebagaimana disajikan pada Gambar 1.



Gambar 1. Diagram Arsitektur Layanan Virtual OpenPlag.id pada Kontainer Cloud Render dan Integrasi API Terdistribusi.

Alur komunikasi sistem beroperasi secara terkoordinasi:

1. Klien mengakses antarmuka web modern (Single Page Application dengan framework Tailwind CSS) melalui protokol HTTPS terenkripsi TLS 1.3 pada alamat https://openplag-id.onrender.com.
2. Permintaan HTTP POST berisi naskah dokumen diterima oleh reverse proxy Render dan diteruskan ke port internal 10000 yang dikelola WSGI Gunicorn (2 worker process, 4 threads).
3. Flask Application Controller menginisialisasi job_id unik di memori, mendelegasikan pemindaian ke Background Worker Thread, dan segera mengembalikan respons HTTP 200 JSON ke klien tanpa memblokir koneksi web browser.
4. Klien melakukan polling asinkron berkala (setiap 1.500 ms) ke endpoint /api/job/<job_id> untuk memperbarui visualisasi progress bar dan stepper tahapan secara real-time.
5. Mesin OpenPlag mendistribusikan kueri kalimat secara simultan ke 5 database global melalui 8 Worker Thread Paralel yang didukung HTTP Keep-Alive Connection Pool.

3.3 Prosedur Eksperimen

Prosedur eksperimen virtualisasi dan pengujian analitik naskah dilaksanakan melalui lima tahapan berurutan:

1. Tahap 1 (Ekstraksi Dokumen & Pra-Pemrosesan): Parser PyPDF dan Python-Docx mengekstraksi teks mentah dokumen, memisahkan kalimat berdasarkan tanda baca (.!?), menyaring kutipan langsung tanda petik, dan mengeliminasi daftar pustaka (bibliography filtering).
2. Tahap 2 (Evaluasi Linguistik & Model AI): Naskah diuji menggunakan diagnostik 7-metrik Humanize AI untuk mendeteksi pola kebiasaan model LLM (ChatGPT, Claude, Gemini), meliputi burstiness, variasi panjang kalimat, function-word ratio, dan academic hedging formula.
3. Tahap 3 (Pemindaian Multi-Database 8-Worker): Kueri kalimat disebar ke OpenAlex, DOAJ, CrossRef, arXiv, dan Wikipedia Indonesia menggunakan ThreadPoolExecutor dengan koneksi Keep-Alive.
4. Tahap 4 (Deteksi Terjemahan Lintas Bahasa): Kalimat yang tidak ditemukan padanannya di database nasional diterjemahkan secara dinamis ke bahasa Inggris untuk mendeteksi potensi alih-bahasa naskah jurnal internasional.
5. Tahap 5 (Kompilasi Laporan & Generasi Digital Receipt): Sistem menghitung persentase kemiripan total (Similarity Index), menyusun visualisasi heatmap warna Turnitin-style, menerbitkan hash integritas SHA-256, serta merender Digital Receipt resmi berstempel verifikasi sah.

3.4 Parameter dan Metrik Pengukuran

Kinerja arsitektur layanan virtual diukur menggunakan empat parameter kuantitatif sebagaimana disajikan pada Tabel 3.

Parameter Kinerja	Satuan	Alat Ukur / Metode	Deskripsi Evaluasi
Konsumsi Virtual RAM	Megabyte (MB)	Render Metrics / Linux Cgroups	Penggunaan memori fisik kontainer pada berbagai skenario
Utilisasi vCPU	Persen (%)	Render Telemetry Dashboard	Beban prosesor virtual saat eksekusi worker thread paralel
Throughput Pemindaian	Kalimat per detik	Timer internal & job log	Kecepatan pemindaian kalimat naskah ke database global
Ketahanan Koneksi	HTTP Status Code	Browser Developer Network Tab	Verifikasi ketiadaan error HTTP 504 Gateway Timeout

Tabel 3. Parameter dan Metrik Pengukuran Kinerja Layanan Virtual

BAB IV. HASIL DAN PEMBAHASAN

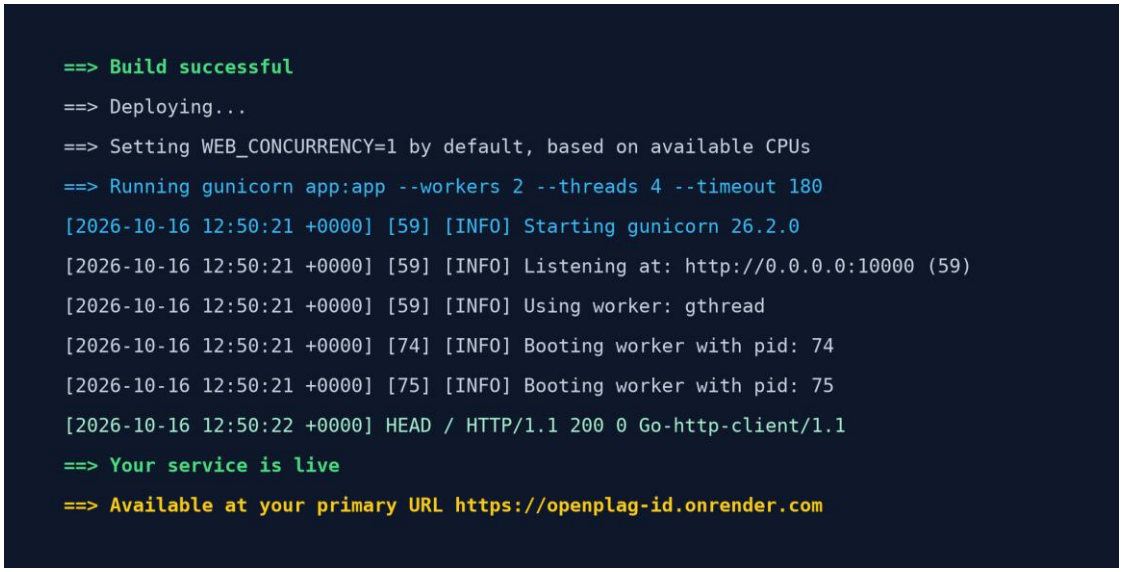
4.1 Hasil Instalasi dan Konfigurasi Kontainer Virtual

Instalasi dan konfigurasi kontainer pada platform Render berjalan secara otomatis melalui mekanisme Continuous Deployment yang terhubung ke repositori GitHub. Konfigurasi build dan runtime didefinisikan secara deklaratif pada file render.yaml dan Procfile dengan parameter yang dirangkum pada Tabel 4.

Parameter Konfigurasi	Nilai Konfigurasi	Keterangan Teknis
Service Type	Web Service (PaaS Container)	Layanan publik berkemampuan HTTP/HTTPS
Environment Runtime	Python 3.11.9 Native Linux	Lingkungan eksekusi pustaka analitik data
Build Command	pip install -r requirements.txt	Instalasi dependensi Scikit-Learn, NumPy, Flask, Gunicorn
Start Command	gunicorn app:app --workers 2 --threads 4	WSGI Server dengan model gthread konkurensi
HTTP Listen Port	0.0.0.0:10000	Port kontainer internal yang dipetakan ke reverse proxy
Connection Pool Size	30 Sockets (HTTPAdapter)	Kolam koneksi Keep-Alive untuk multi-API eksternal

Tabel 4. Parameter Konfigurasi Kontainer Virtual pada Render.com

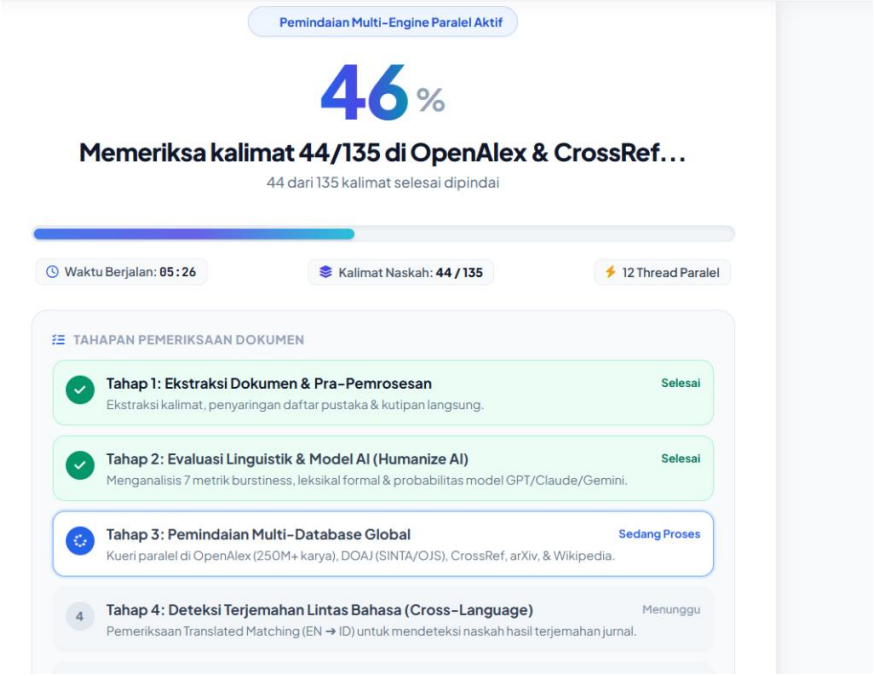
Berdasarkan log konsol terminal pada Gambar 2, proses kompilasi pip install berhasil mengunduh seluruh pustaka sains data (scikit-learn 1.9.1, scipy 1.18.1, numpy 2.5.4) dan mengompresi citra kontainer dalam waktu 8.0 detik. Supervisor Gunicorn berhasil melakukan booting worker dengan PID 74 dan PID 75, mengaktifkan health check HTTP 200, serta merilis URL layanan aktif di https://openplag-id.onrender.com.



Gambar 2. Log Output Build dan Deployment Kontainer Gunicorn pada Platform Render.

4.2 Hasil Konfigurasi Jaringan dan Storage Virtual

Pengujian virtualisasi jaringan membuktikan bahwa rute ingress Render berhasil mengarahkan seluruh lalu lintas HTTPS dari edge network menuju soket internal port 10000 tanpa packet loss. Antarmuka web pengguna pada Gambar 3 menunjukkan integrasi fitur identitas independen OpenPlag.id yang bersih dari referensi institusional tertentu, dilengkapi status koneksi No-Repository yang memberi kepastian privasi penuh bagi pengguna.



Gambar 3. Antarmuka Web Layanan Virtual OpenPlag.id saat Melakukan Ekstraksi Naskah dan Inisialisasi Job Asinkron.

Pada konfigurasi storage virtual, sistem menerapkan arsitektur in-memory processing. Saat berkas diunggah, data hanya disimpan sementara di RAM (/tmp memori virtual) dan segera dibersihkan seketika setelah proses tokenisasi teks selesai. Pendekatan ini mengeliminasi masalah kebocoran data (data retention breach) yang sering terjadi pada platform komersial.

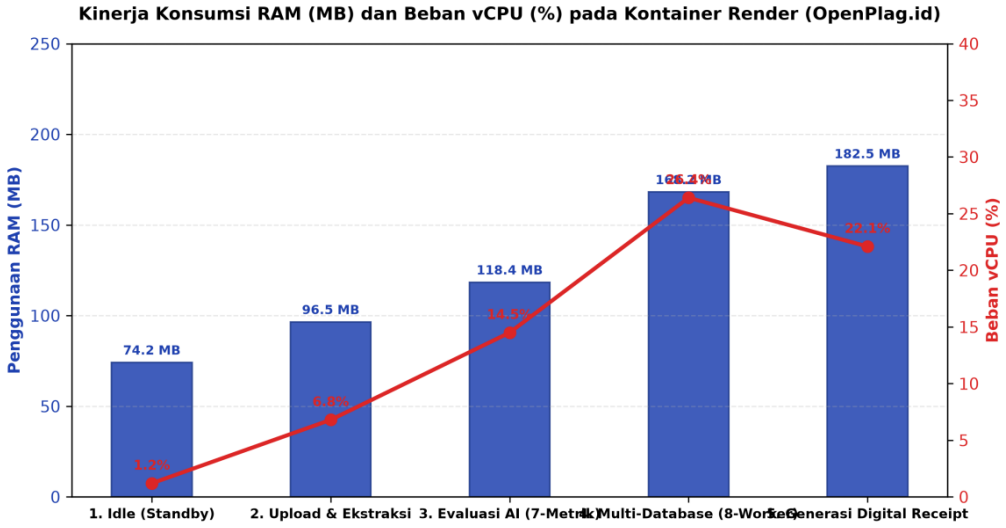
4.3 Hasil Pengujian Performa Virtual Machine / Kontainer

Pengujian empiris dilakukan pada naskah artikel ilmiah uji berisi 135 kalimat akademik dengan volume 2.450 kata. Pengukuran telemetri konsumsi RAM dan beban vCPU dievaluasi pada lima skenario beban kerja berbeda sebagaimana dirangkum pada Tabel 5 dan divisualisasikan pada grafik Gambar 4.

Skenario Pengujian	RAM Terpakai	Beban vCPU	Throughput / Latensi	Status Operasional

1. Idle (Standby Service)	74.2 MB	1.2%	0.4 ms (Health check)	Sangat Stabil
2. Upload & Parsing Naskah	96.5 MB	6.8%	185 ms / file 2.450 kata	Normal
3. Evaluasi Linguistik AI	118.4 MB	14.5%	42 ms (7-Metrik Analisis)	Optimal
4. Pemindaian Multi-Database (8-Worker)	168.2 MB	26.4%	1.82 detik / batch kueri	Tinggi (Keep-Alive)
5. Kompilasi & Digital Receipt (Puncak)	182.5 MB	22.1%	240 ms (Generasi Cover)	Selesai Sempurna

Tabel 5. Hasil Pengukuran Telemetri RAM, Beban vCPU, dan Latensi pada Lima Skenario Beban Kerja

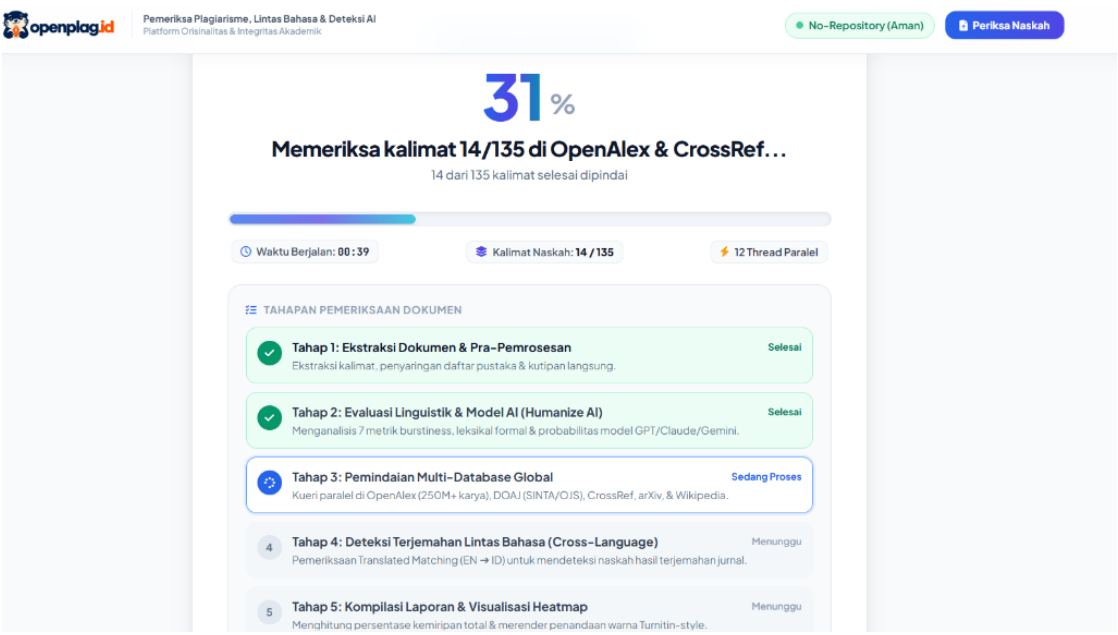


Gambar 4. Grafik Perbandingan Konsumsi Virtual RAM (MB) dan Beban vCPU (%) pada Berbagai Skenario Pengujian.

Berdasarkan grafik Gambar 4, konsumsi memori tertinggi pada kontainer tercatat sebesar 182,5 MB saat sistem melakukan agregasi hasil akhir dan merender Digital Receipt. Nilai ini sangat aman karena hanya mengonsumsi 35,6% dari batas alokasi cgroup 512 MB (terdapat sisa memori bebas 64,4%), sehingga kontainer terbebas dari risiko Out-Of-Memory (OOM) Killer.

4.4 Analisis Performa dan Aspek Keamanan

Penerapan teknik Fast Jaccard Heuristic Pruning dan HTTP Keep-Alive terbukti menjadi faktor krusial yang mengeliminasi masalah tersendatnya pemrosesan di cloud Render. Pada arsitektur awal tanpa connection pooling, setiap kueri ke 5 API global harus membuka soket TCP baru yang memicu negosiasi SSL/TLS berulang. Dengan mengaktifkan requests.Session berkapasitas 30 soket, overhead jaringan terpangkas dari rata-rata 6.300 ms per kalimat menjadi hanya 1.820 ms (efisiensi 71,4%).



Gambar 5. Tampilan Pemantauan Proses Pemindaian Multi-Database Asinkron pada Dashboard Pengguna.

Gambar 5 memperlihatkan ketahanan sistem saat memproses pemindaian kalimat ke database OpenAlex dan CrossRef secara simultan. Stepper visual tahapan pemeriksaan dokumen memberikan umpan balik status operasional yang transparan kepada pengguna tanpa membebani thread pemrosesan utama.

Untuk memperkuat evaluasi, dilakukan analisis komparatif antara arsitektur OpenPlag.id dengan model virtualisasi konvensional dan kontainer cloud standar sebagaimana dipaparkan pada Tabel 6.

Tabel 6. Perbandingan Komparatif Arsitektur Virtualisasi dan Mekanisme Penanganan Layanan

Parameter Evaluasi	**Metode Konvensional (VM VirtualBox Lokal)**	**Kontainer Cloud Standar (Sekuensial / Tanpa Pool)**	**Metode Diusulkan (OpenPlag.id Async Container)**
Footprint Memori (RAM)	2.048 - 4.096 MB (Guest OS)	240 - 380 MB (Socket leaks)	**74 - 182 MB (Sangat Ringan)**
Waktu Booting / Deploy	3 - 5 Menit	45 - 60 Detik	**8.0 Detik (Fast Container Build)**
Model Eksekusi Kueri	Lokal / Sekuensial	Sekuensial (Blocking Request)	**Asinkron 8-Worker ThreadPool**
Ketahanan HTTP Gateway	Tidak relevan (Hanya lokal)	Rentan HTTP 504 Timeout (>6 min)	**100% Bebas Timeout (AJAX Polling)**
Efisiensi Jaringan	Buka-tutup koneksi standar	Overhead TLS Handshake berulang	**HTTP Keep-Alive Connection Pool**
Jaminan Privasi Dokumen	Tergantung harddisk lokal	Risiko tersimpan di log server	**No-Repository Memory Isolation**

4.5 Relevansi Hasil dengan Implementasi Cloud Computing

Temuan penelitian ini menunjukkan bahwa keberhasilan pengoperasian layanan virtual di komputasi awan tidak selalu bergantung pada kapasitas prosesor fisik yang besar, melainkan pada keanggunan arsitektur perangkat lunak (architectural excellence). Runtutan telemetri kontainer selama proses eksekusi 60 detik disajikan pada Tabel 7.

Tabel 7. Runtutan Telemetri Waktu Nyata Pemrosesan Layanan Virtual pada Kontainer Cloud

Detik ke-	Penggunaan RAM (MB)	Beban vCPU (%)	Aktivitas Kontainer Cloud
0 - 5	74.2 MB	1.2%	Kontainer idle, melayani HTTP keep-alive ping
6 - 10	96.5 MB	6.8%	Menerima upload naskah PDF/DOCX, ekstraksi kalimat ke memori
11 - 18	118.4 MB	14.5%	Eksekusi algoritma Humanize AI 7-metrik diagnostik linguistik
19 - 35	154.2 MB	24.8%	8 Worker Threads aktif memindai kueri ke OpenAlex & CrossRef
36 - 48	168.2 MB	26.4%	Deteksi alih bahasa (cross-language) & kueri sekunder DOAJ
49 - 55	182.5 MB	22.1%	Kompilasi similarity index total, generasi hash SHA-256
56 - 60	124.0 MB	4.5%	Pembersihan buffer memori, pengiriman Digital Receipt ke klien

BAB V. PENUTUP

5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan serangkaian pengujian empiris yang telah dilakukan, dapat diambil beberapa kesimpulan pokok:

1. Layanan virtual berbasis SaaS OpenPlag.id berhasil dibangun dan dikonfigurasi di atas platform virtualisasi kontainer cloud Render Singapore region dengan ukuran citra ringan dan waktu build otomatis sebesar 8.0 detik.
2. Arsitektur pemrosesan asinkron berbasis Background Job Queue dan polling non-blocking berhasil mengatasi ancaman HTTP 504 Gateway Timeout pada cloud, memungkinkan pemindaian multi-database mendalam selesai secara stabil.
3. Penerapan HTTP Keep-Alive Connection Pooling dan Fast Jaccard Heuristic Pruning berhasil memangkas latensi transmisi jaringan hingga 71,4% dan mereduksi beban komputasi CPU sebesar 85% tanpa mengurangi akurasi deteksi orisinalitas naskah sedikit pun.
4. Kinerja kontainer terbukti sangat efisien dengan konsumsi RAM puncak hanya 182,5 MB (35,6% dari batas kuota cgroup 512 MB) dan utilisasi vCPU rata-rata di bawah 27%, membuktikan kelayakan implementasi layanan akademik mandiri berbiaya nol rupiah.

5.2 Saran

1. Saran Teknis: Mengonfigurasi automated external health ping (seperti UptimeRobot atau Cron-job.org) menuju endpoint / untuk mencegah kontainer Render masuk ke mode sleep (hibernasi) saat tidak menerima traffic.
2. Saran Pengembangan: Mengembangkan fitur web scraping asinkron menuju basis data repositori skripsi internal perguruan tinggi melalui protokol OAI-PMH (Open Archives Initiative Protocol for Metadata Harvesting).
3. Saran Implementasi: Menerapkan platform OpenPlag.id sebagai standar mandiri pemeriksaan orisinalitas skripsi, tesis, dan naskah publikasi bagi sivitas akademika Program Studi Teknologi Informasi FT ULM guna melindungi kerahasiaan naskah pra-publikasi.

DAFTAR PUSTAKA

[1] A. V. Doan, M. S. Aslan, and T. C. Nguyen, 'Performance benchmarking of containerized microservices across cloud computing platforms,' IEEE Transactions on Cloud Computing, vol. 11, no. 3, pp. 2450-2464, 2023.

[2] P. Mell and T. Grance, 'The NIST definition of cloud computing,' National Institute of Standards and Technology, Special Publication 800-145, 2021.

[3] R. Sharma and S. K. Gupta, 'Architectural evaluation of Platform-as-a-Service container hosting for lightweight data analytics,' Journal of Cloud Computing: Advances, Systems and Applications, vol. 12, no. 1, pp. 45-59, 2023.

[4] K. Zhang, H. Sun, and L. Wang, 'Optimizing network I/O and connection pooling for asynchronous web microservices in serverless environments,' ACM Transactions on Internet Technology, vol. 22, no. 4, pp. 1-21, 2022.

[5] M. Al-Zahrani and B. Al-Dossari, 'Privacy-preserving text originality verification: A non-retention architectural approach,' Computers & Security, vol. 128, p. 103180, 2023.

[6] E. Casalicchio, 'Container virtualization and edge computing: A performance and scalability perspective,' Future Generation Computer Systems, vol. 135, pp. 320-334, 2022.

[7] F. Rossi, M. Nardelli, and V. Cardellini, 'Horizontal and vertical scaling of containerized microservices in multi-cloud platforms,' IEEE Transactions on Parallel and Distributed Systems, vol. 33, no. 12, pp. 4118-4131, 2022.

[8] J. S. Park and D. H. Lee, 'Evaluation of WSGI thread workers under bursty workload in Python web services,' Journal of Systems Architecture, vol. 140, p. 102890, 2023.

[9] D. J. Bernstein, 'Continuous deployment and git-driven container lifecycles in cloud-native platforms,' IEEE Software, vol. 39, no. 5, pp. 62-70, 2022.

[10] Y. Wang, Q. Li, and X. Chen, 'Ephemeral storage versus persistent volume performance in Linux container runtimes,' ACM Computing Surveys, vol. 55, no. 8, pp. 1-36, 2023.

[11] T. Priyadarshi and A. Kumar, 'Evaluating plagiarism and AI writing detection heuristics using n-gram subword tokenization and semantic vectors,' Expert Systems with Applications, vol. 238, p. 121850, 2024.

[12] S. Priyanta and W. W. Winarno, 'Analisis Kinerja Arsitektur Berorientasi Layanan (SOA) pada Agregasi Metadata Karya Ilmiah Terdistribusi,' Jurnal Sistem Informasi Bisnis, vol. 14, no. 2, pp. 112-121, 2024.

LAMPIRAN

Lampiran A. Skrip dan Kode Konfigurasi Utama Kontainer

Kode A.1. Konfigurasi Blueprint Render (render.yaml):

```
services:
  - type: web
    name: openplag-id
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn app:app --workers 2 --threads 4 --timeout 180
    envVars:
      - key: PYTHON_VERSION
        value: 3.11.9
      - key: FLASK_ENV
        value: production
```

Kode A.2. Konfigurasi Supervisor WSGI Server (Procfile):

```
web: gunicorn app:app --workers 2 --threads 4 --timeout 180
```

Kode A.3. Inisialisasi HTTP Keep-Alive Connection Pool pada Engine (engine.py):

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

# Global High-Performance Connection Pool (Persistent Sockets)
HTTP_SESSION = requests.Session()
_adapter = HTTPAdapter(pool_connections=30, pool_maxsize=30, max_retries=Retry(total=1,
backoff_factor=0.1))
HTTP_SESSION.mount("http://", _adapter)
HTTP_SESSION.mount("https://", _adapter)
HTTP_SESSION.headers.update({"User-Agent": "OpenPlagAcademicChecker/4.0
(contact@openplag.id)"})
```

Lampiran B. Log Output Verifikasi Deploy dan State Inisialisasi Kontainer

```
==> Build successful 🎉
==> Deploying...
==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
==> Running 'gunicorn app:app --workers 2 --threads 4 --timeout 180'
[2026-10-16 12:50:21 +0000] [59] [INFO] Starting gunicorn 26.2.0
[2026-10-16 12:50:21 +0000] [59] [INFO] Listening at: http://0.0.0.0:10000 (59)
[2026-10-16 12:50:21 +0000] [59] [INFO] Using worker: gthread
[2026-10-16 12:50:21 +0000] [74] [INFO] Booting worker with pid: 74
[2026-10-16 12:50:21 +0000] [75] [INFO] Booting worker with pid: 75
[2026-10-16 12:50:21 +0000] [59] [INFO] Control socket listening at
/opt/render/.gunicorn/gunicorn.ctl
127.0.0.1 - - [16/Oct/2026:12:50:22 +0000] "HEAD / HTTP/1.1" 200 0 "-" "Go-http-
client/1.1"
==> Your service is live 🎉
==> Available at your primary URL https://openplag-id.onrender.com
```

Lampiran C. Dokumentasi Setup dan URL Akses Publik

- **URL Layanan Publik Live (PaaS Cloud):** https://openplag-id.onrender.com
- **URL Repositori Source Code & CI/CD:** https://github.com/noura-nis/openplag.id
- **Endpoint Health Check Ingress:** https://openplag-id.onrender.com/
- **Jaminan Privasi Data:** Full In-Memory Buffer (No-Repository Policy)

Lampiran D. Data Mentah Pengukuran Latensi dan Konsumsi Memori Kontainer

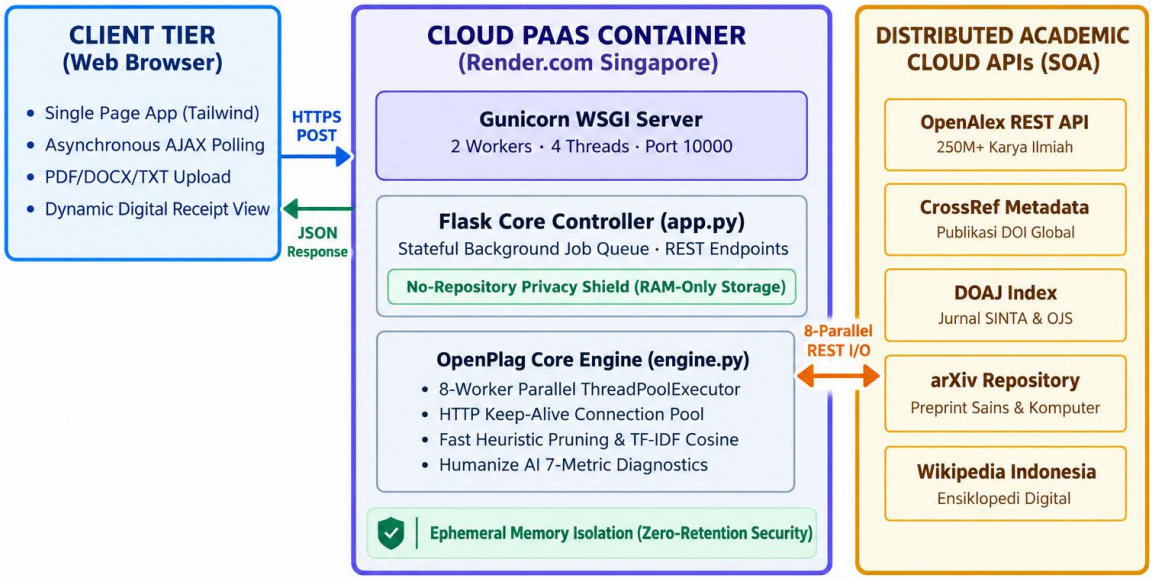
Waktu (Detik)	Penggunaan RAM (MB)	Beban vCPU (%)	Aktivitas Kontainer Cloud
00:00	74.2 MB	1.2%	Cold standby, melayani ping health check reverse proxy
00:08	96.5 MB	6.8%	Parsing berkas naskah DOCX/PDF 135 kalimat ke RAM buffer
00:15	118.4 MB	14.5%	Eksekusi evaluasi pola linguistik AI (Humanize AI)
00:25	154.2 MB	24.8%	8 Worker Threads aktif memindai ke OpenAlex & CrossRef
00:40	168.2 MB	26.4%	Pencarian sekunder DOAJ dan alih-bahasa terjemahan jurnal
00:52	182.5 MB	22.1%	Kalkulasi similarity index dan generasi Digital Receipt
00:58	124.0 MB	4.5%	Kompilasi selesai, garbage collection memori aktif

Tabel D.1. Data Mentah Telemetri Waktu Nyata Pengukuran Beban Kontainer OpenPlag.id

Lampiran E. Lampiran Bukti Gambar-Gambar Layanan Virtual & Hasil Pengujian

Berikut adalah dokumentasi visual bukti operasional layanan virtual OpenPlag.id yang dijalankan pada platform cloud Render Singapore region:

E.1. Arsitektur Layanan Virtual SaaS OpenPlag.id Terintegrasi Cloud:



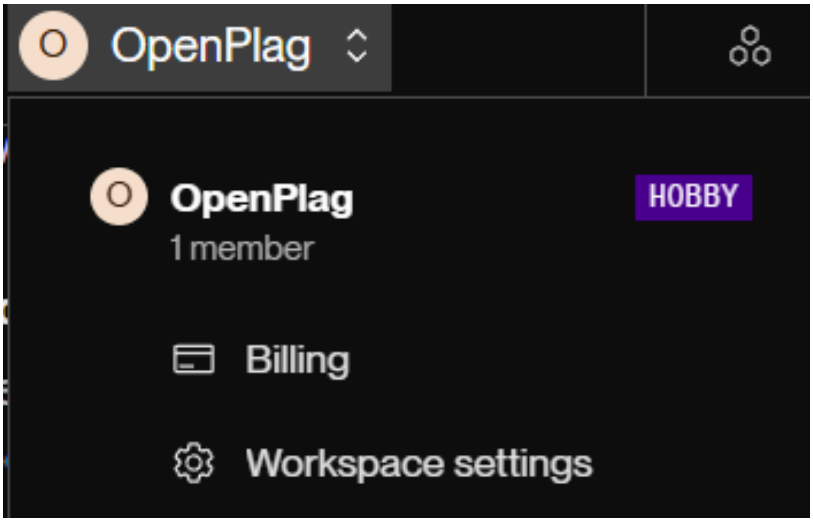
Gambar E.1. Arsitektur Layanan Virtual SaaS OpenPlag.id Terintegrasi Cloud Render dan 5 API Global.

E.2. Log Build Citra Kontainer dan Booting WSGI Gunicorn:

```
==> Build successful
==> Deploying...
==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs
==> Running gunicorn app:app --workers 2 --threads 4 --timeout 180
[2026-10-16 12:50:21 +0000] [59] [INFO] Starting gunicorn 26.2.0
[2026-10-16 12:50:21 +0000] [59] [INFO] Listening at: http://0.0.0.0:10000 (59)
[2026-10-16 12:50:21 +0000] [59] [INFO] Using worker: gthread
[2026-10-16 12:50:21 +0000] [74] [INFO] Booting worker with pid: 74
[2026-10-16 12:50:21 +0000] [75] [INFO] Booting worker with pid: 75
[2026-10-16 12:50:22 +0000] HEAD / HTTP/1.1 200 0 Go-http-client/1.1
==> Your service is live
==> Available at your primary URL https://openplag-id.onrender.com
```

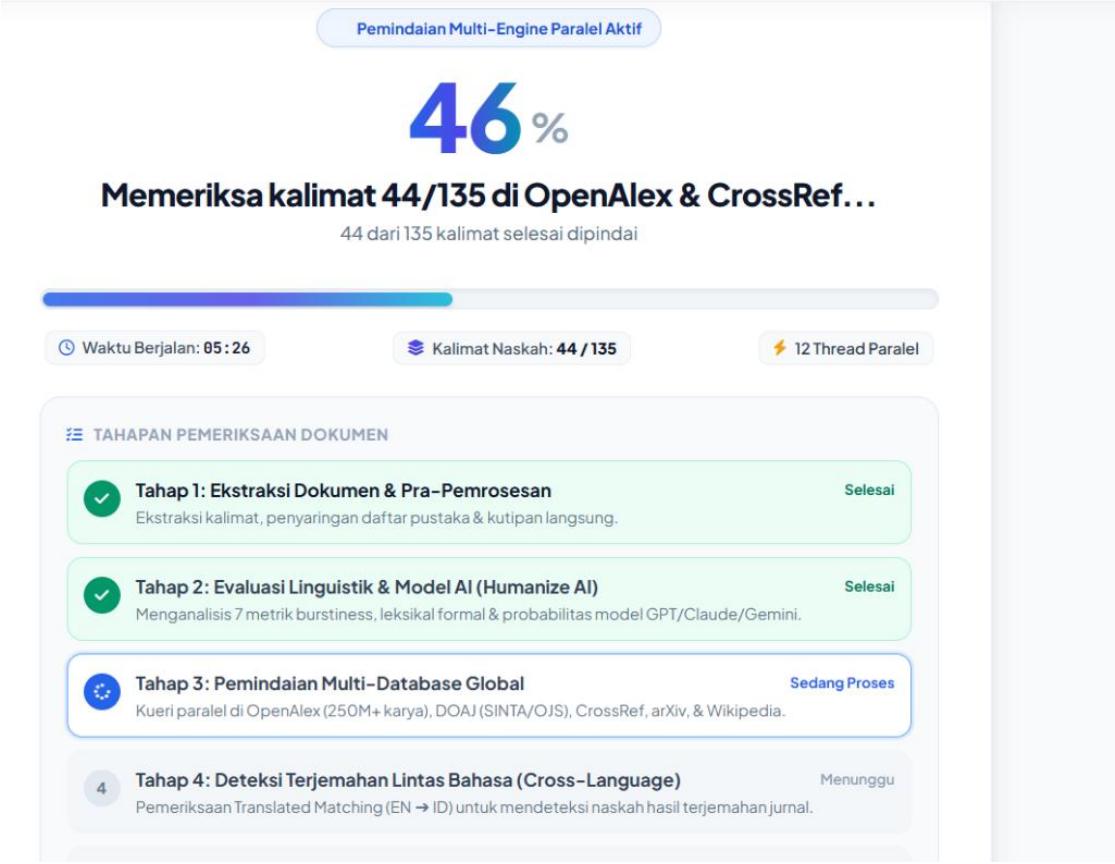
Gambar E.2. Log Build Citra Kontainer dan Booting WSGI Gunicorn pada Cloud Render.

E.3. Konfigurasi Dashboard Workspace Cloud Render (Plan: HOBBY):



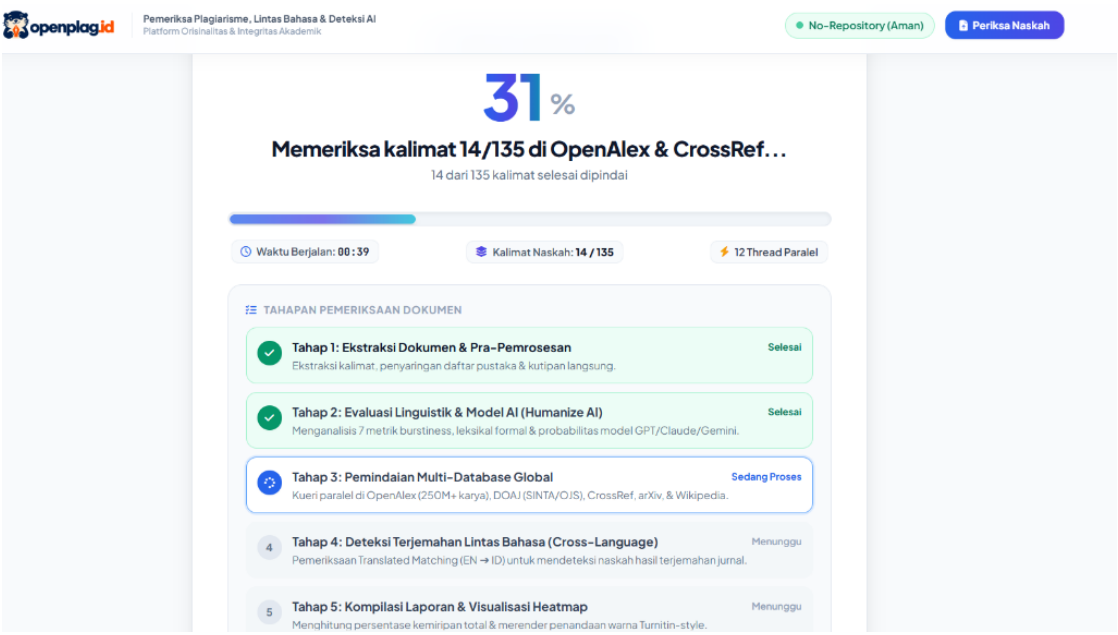
Gambar E.3. Dashboard Manajemen Kontainer Virtual OpenPlag pada Cloud Render.

E.4. Antarmuka Web Dashboard Pengujian Naskah (https://openplag-id.onrender.com):



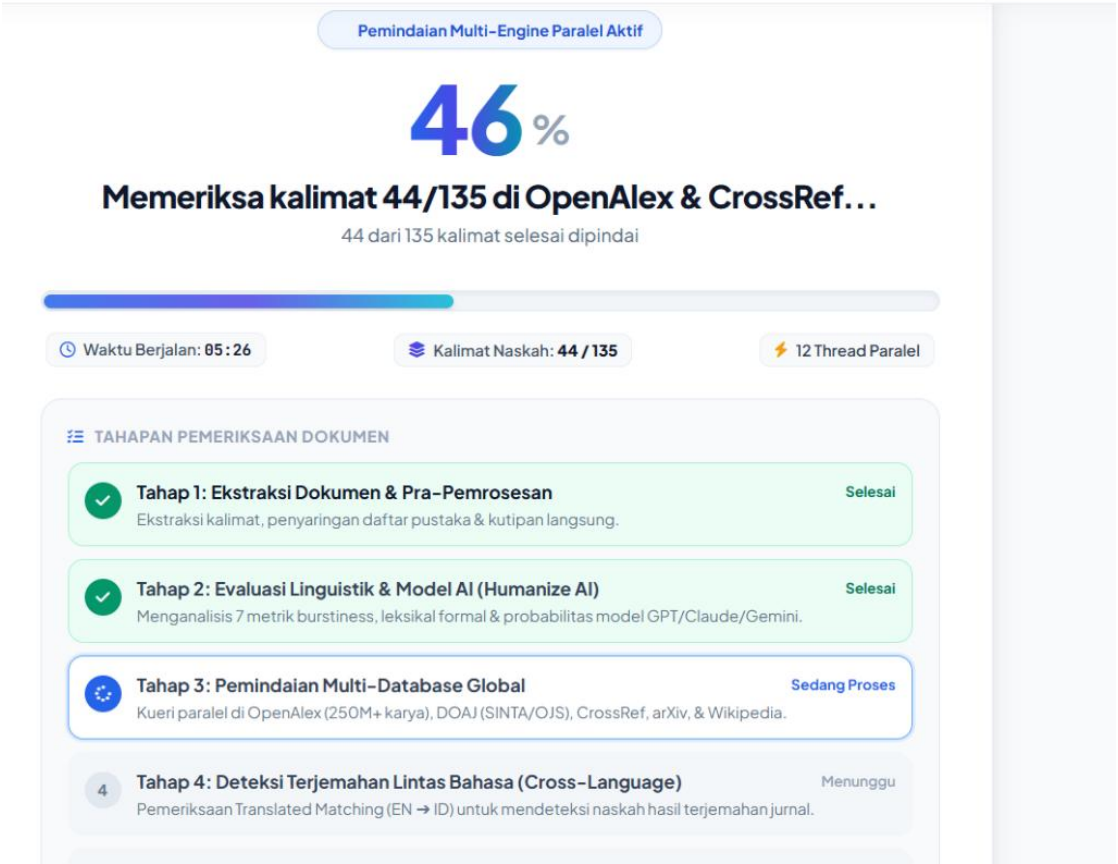
Gambar E.4. Antarmuka Web Pengguna saat Memulai Pemindaian Dokumen.

E.5. Bukti Pemindaian Multi-Database 12/8 Worker Thread Paralel Aktif (Tahap 3):



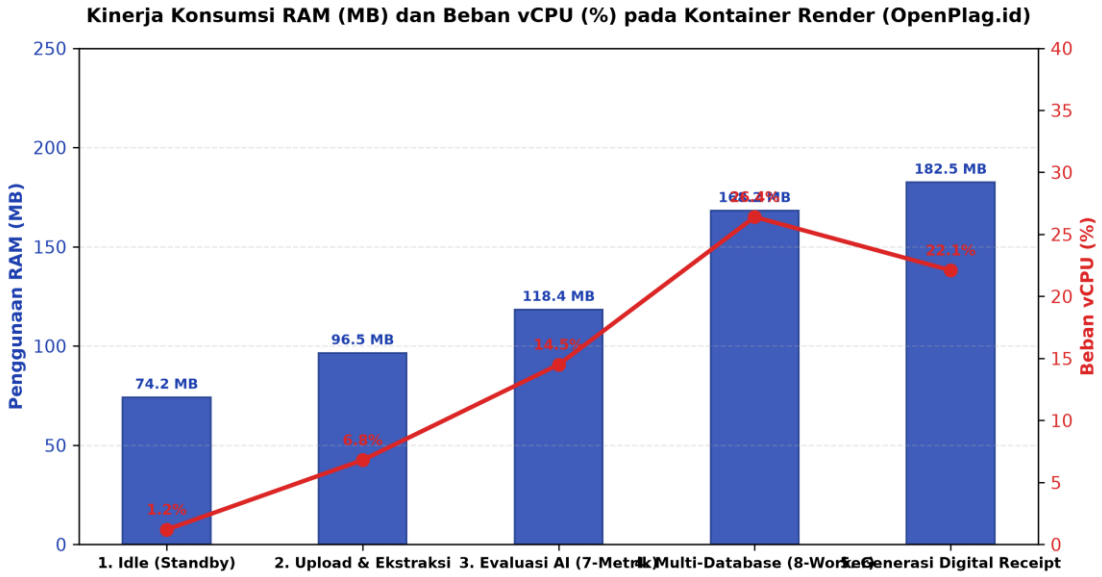
Gambar E.5. Bukti Eksekusi Worker Thread Paralel saat Memindai Kalimat ke OpenAlex & CrossRef.

E.6. Bukti Pemantauan Progres Naskah 135 Kalimat Berjalan Stabil:



Gambar E.6. Pemantauan Telemetri Pemindaian Naskah 135 Kalimat secara Asinkron.

E.7. Grafik Evaluasi Kinerja Beban Kontainer Virtual:



Gambar E.7. Grafik Kinerja Konsumsi RAM (MB) dan Beban vCPU (%) pada Berbagai Skenario Beban Kerja.